

TPB2 : Création d'application avec Flutter

Table des matières

1) Problématique.....	1
-----------------------	---

1) Problématique

L'objectif de ce TP est de concevoir et développer une application à l'aide de Flutter, en utilisant Android Studio et Visual Studio Code comme environnements de développement. Le langage Dart sera utilisé pour programmer l'application.

2) Prérequis

Afin de mener à bien ce TP nous allons avoir besoin de 4 outils :

- **Visual Studio Community 2026** :

<https://visualstudio.microsoft.com/fr/vs/community/>

- **Visual Studio Code** : <https://code.visualstudio.com/Download>

- **SDK Flutter** : <https://docs.flutter.dev/install/manual>

- **Android Studio** : <https://developer.android.com/studio>

Il va falloir installer chaque outil et configurer certains. Par exemple, il faudra ajouter le kit de développement C++ pour Visual Studio Community et il faudra installer l'outil CMD Android Tools.

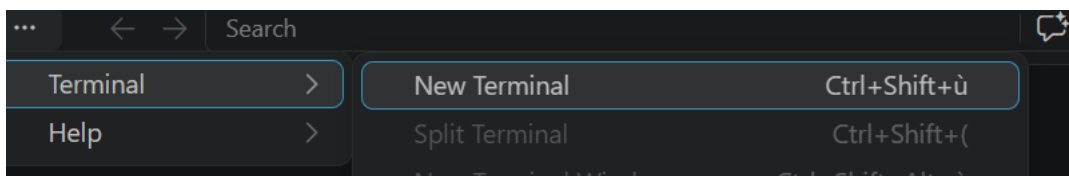
On peut vérifier grâce à la commande flutter doctor si tout est bien configuré :

```
Microsoft Windows [version 10.0.26200.9278]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\Ewan Alexandre>flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.47.2, on Microsoft Windows [version 10.0.26200.9278], locale fr)
[✓] Windows Version (Windows 11 or higher, 25H2, 2009)
[✓] Android toolchain - develop for Android devices (Android SDK version 36.0.0)
[✓] Chrome - develop for the web
[✓] Visual Studio - develop Windows apps (Visual Studio Community 2026 18.9.2)
[✓] Connected device (3 available)
[✓] Network resources

• No issues found!
```

Dans VSCode, on commence d'abord par ouvrir le terminal comme ceci :



Ensuite, dans le terminal on se place sur le dossier du projet avec **cd** :

```
PS C:\Users\Ewan Alexandre> cd develop
PS C:\Users\Ewan Alexandre\develop>
```

Ensuite on utilise **flutter create nom_application** pour créer la base de notre application :

```
PS C:\Users\Ewan Alexandre\develop> flutter create mon_app
Creating project mon_app...
Resolving dependencies in `mon_app`...
Downloading packages...
Got dependencies in `mon_app`.
Wrote 131 files.

All done!
```

Puis cette fois-ci on se place dans le dossier de l'application avec **cd** puis on utilise **code .** pour ouvrir le projet dans VSCode :

```
PS C:\Users\Ewan Alexandre\develop> cd mon_app
PS C:\Users\Ewan Alexandre\develop\mon_app> code .
```

Une page devrait alors s'ouvrir, avec le projet créé ouvert. On va chercher le main.dart dans le dossier lib de cette manière :



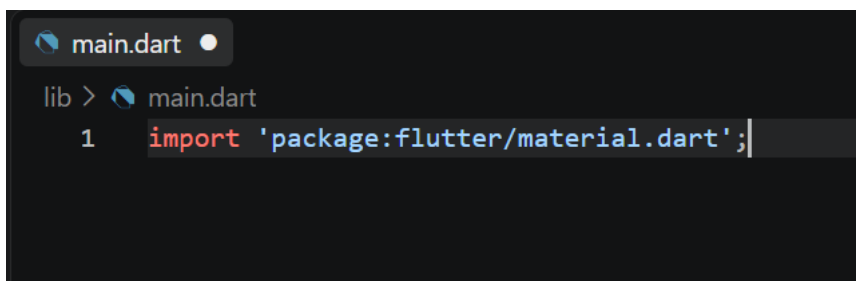
Nous allons désormais supprimer son contenu par défaut pour pouvoir commencer à créer notre application de zéro.

3) Exercices (Flutter)

1. Page vide

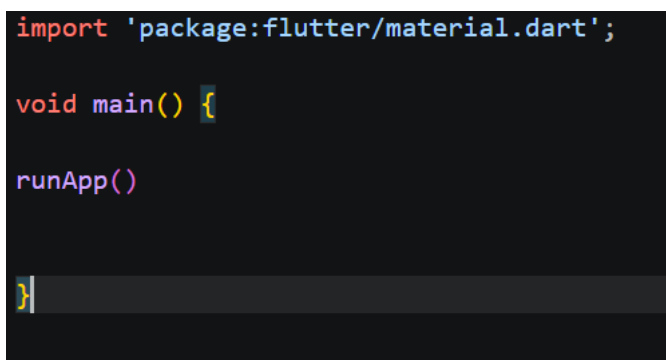
Nous pouvons à présent commencer à travailler sur notre projet, notre première étape sera de créer une page vide.

Pour créer une page vide on s'aide du guide qui nous est donné, celui-ci nous dit que l'on doit d'abord importer le package Material de Flutter comme ceci :



```
main.dart
lib > main.dart
1 import 'package:flutter/material.dart';
```

Par la suite on nous demande d'ajouter le **void main()** qui permettra d'exécuter le code, et à l'intérieur de celui-ci on ajoute la fonction **runApp()** de cette manière :



```
import 'package:flutter/material.dart';

void main() {

  runApp()

}
```

Parfait ! Nous avons maintenant une base qui permettra d'effectuer la suite du TP.

2. Créer un fond de couleur

Pour pouvoir créer un fond de couleur pour notre application nous allons utiliser le widget **Container()** qui représente une boîte simple. Nous allons attribuer la couleur rouge au container avec le paramètre « color : » puis on

sélectionne l'attribut « red » de la classe « Colors » avec Colors.red et on attribue la classe au paramètre colors : comme ceci :

```
Container(  
  color: Colors.red,  
),
```

Les , sont l'équivalent des ; elles permettent d'indiquer la fin du ligne au sein d'un argument donc color est attribué à container et la , après container indique que celui-ci est fini. Le code entier ressemble à ceci :

```
import 'package:flutter/material.dart';  
  
Run | Debug | Profile  
void main() {  
  runApp(  
    Container(  
      color: Colors.red,  
    ),  
  );  
}
```

Et le résultat est le suivant :



3. Créer du texte centré

Pour créer un texte centré nous allons utiliser le paramètre **child** pour ajouter un enfant au Container, cet enfant sera un widget **Center**, de type **const**, qui lui-même aura pour enfant un **Text** avec un paramètre de direction qui précise que le texte va de gauche à droite (« ltr ») :

```

Run | Debug | Profile
void main() {
  runApp(
    Directionality(
      textDirection: TextDirection.ltr,
      child: Container(
        color: Colors.red,
        child: const Center(
          child: Text(
            'Hello, World!',
          ), // Text
        ), // Center
      ), // Container
    ), // Directionality
  );
}

```

4) Exercices (DART)

Pour effectuer les exercices DART, nous allons utiliser VCC pour créer notre code et le tester et les documentations DART pour nous aider à répondre aux questions.

Exercice 1 :

On commence par importer l'outil permettant de capter les entrées claviers : « dart:io ». On demande à l'utilisateur son nom puis celui-ci pourra l'indiquer et la réponse sera contenu dans la variable nom. Pareil pour ageTexte, cependant ageTexte est considéré comme un texte on utilise donc un int.parse pour convertir le texte en entier puis on attribue le résultat à une nouvelle variable age. Puis on déclare une variable anneesRestantes qui calcule combien d'années il reste à l'utilisateur avant qu'il ait 100 ans. A la fin on affiche un message qui lui indique donc le résultat et on utilise \$ pour afficher le contenu des variables.

```

import 'dart:io';

Run | Debug | Profile
void main() {
  print("Entrez votre nom :");
  String nom = stdin.readLineSync();

  print("Entrez votre âge :");
  String ageTexte = stdin.readLineSync();

  int age = int.parse(ageTexte);
  int anneesRestantes = 100 - age;

  print("Bonjour $nom, il te reste $anneesRestantes ans avant d'avoir 100 ans !");
}

```

Ensuite on lance le code avec **dart run lib/main.dart**.

Résultat :

```
Entrez votre nom :  
Ewan  
Entrez votre âge :  
19  
Bonjour Ewan, il te reste 81 ans avant d'avoir 100 ans !
```

Exercice 2 :

Ici on demande à l'utilisateur d'entrer un nombre, on récupère ce nombre et on le stocke dans la variable `saisie`, ensuite on convertit ce « faux » nombre en vrai nombre avec le `int.parse`. Par la suite on effectue un test pour vérifier si la division du nombre choisit par 2 laisse un reste de 0 ou non, si oui alors on indique que c'est pair, sinon on indique que c'est impair.

```
import 'dart:io';  
  
Run | Debug | Profile  
void main() {  
  print("Entrez un nombre :");  
  String saisie = stdin.readLineSync()!;  
  
  int nombre = int.parse(saisie);  
  
  if (nombre % 2 == 0) {  
    print("Le nombre $nombre est pair.");  
  } else {  
    print("Le nombre $nombre est impair.");  
  }  
}
```

Résultats :

```
Entrez un nombre :  
20  
Le nombre 20 est pair.
```

```
Entrez un nombre :  
19  
Le nombre 19 est impair.
```

Exercice 3 :

On crée une liste a, on fait une boucle qui vérifie si le nombre de chaque indice est strictement inférieur à 5, si oui alors le nombre est affiché, sinon rien n'est affiché. Cela affichera donc que les éléments inférieurs à 5.

```
void main() {  
    List<int> a = [1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89];  
  
    for (int nombre in a) {  
        if (nombre < 5) {  
            print(nombre);  
        }  
    }  
}
```

Résultat :

```
PS  
1  
1  
2  
3
```

Exercice 4 :

Ici on demande un nombre à l'utilisateur, on convertit sa réponse en entier, on fait une boucle qui part de 1 (i est le compteur) et qui ne s'arrête pas tant que la condition i inférieur ou égal au nombre choisi par l'utilisateur est vrai puis i augmente de 1 à chaque réitération avec l'incrémementation i++.

Ensuite on vérifie que le nombre choisi par l'utilisateur divisé par i donne un reste de 0. Si c'est le cas alors i est un diviseur du nombre choisi et donc on l'affiche, sinon on l'affiche pas.

```
import 'dart:io';  
  
Run | Debug | Profile  
void main() {  
    print("Entrez un nombre :");  
    String saisie = stdin.readLineSync();  
  
    int nombre = int.parse(saisie);  
  
    for (int i = 1; i <= nombre; i++) {  
        if (nombre % i == 0) {  
            print(i);  
        }  
    }  
}
```

Résultat :

```
Entrez un nombre :  
10  
1  
2  
5  
10
```

Exercice 5 :

On déclare les deux listes, on crée une liste qui va récupérer les éléments communs des deux listes. On effectue une boucle dans une boucle ce qui va permettre de vérifier chaque indice un par un de chaque liste en même temps. Le `for (int nombreA in a)` est une boucle `for each` donc ça parcourt tous les éléments de la liste. Pareil pour le B du coup. Dans la boucle du B on vérifie donc que le nombre présent à l'indice de A est le même que celui présent au même endroit à l'indice de B. Si c'est le cas alors le nombre est ajouté à la liste « communs », si ça ne l'est pas alors rien ne se passe. Puis lorsque tous les indices ont été parcourus les deux boucles s'arrêtent et il ne reste plus qu'à afficher la liste des nombres communs. Une vérification se fait pour voir si un nombre en commun entre les deux listes n'est pas déjà dans la liste commune. Par exemple, imaginons qu'à l'indice 4 des deux boucles le nombre est 5, et que c'est aussi le cas pour l'indice 5, alors dans ce cas cela ne va pas ajouter deux fois le nombre 5 dans la liste des communs.

```
void main() {  
    List<int> a = [1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89];  
    List<int> b = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13];  
  
    List<int> communs = [];  
  
    for (int nombreA in a) {  
        for (int nombreB in b) {  
            if (nombreA == nombreB) {  
                if (!communs.contains(nombreA)) {  
                    communs.add(nombreA);  
                }  
            }  
        }  
    }  
  
    print(communs);  
}
```

Exercice 6 :

Ici, la logique est plus difficile. En gros on demande un mot à l'utilisateur. On déclare une variable `chaineInverse` vide qui va permettre de stocker lettre par lettre le mot inversé. Ensuite, on déclare le compteur qui prendra la valeur de la longueur de la chaîne (par exemple `kayak = 5` donc `i = 5`) et enlève -1 car en fait ce sont les indices qui vont être parcourus donc si on commençait à 5 le code ne serait pas quoi faire car `kayak` va de 0 à 4 en termes d'indices. On décide donc d'enlever 1 au tout pour que ça commence bien avec la dernière lettre puis on décrémente jusqu'à que la condition `i >= 0` soit fausse. On fait supérieur ou égal à 0 pour aussi prendre l'indice 0 du nombre (donc la première lettre). Enfin, à chaque boucle la lettre présente à l'indice `i` est ajoutée à la variable `chaineInverse`, et une vérification `if` se fait pour vérifier que la chaîne de base est bien égale à la chaîne inverse, si c'est le cas alors ça indique la chaîne est un palindrome sinon elle indique que ça n'en est pas un.

```
Run | Debug | Profile
✓ void main() {
    print("Entrez une chaîne :");
    String chaine = stdin.ReadLineSync();

    String chaineInverse = "";
    ✓ for (int i = chaine.Length - 1; i >= 0; i--) {
        |     chaineInverse = chaineInverse + chaine[i];
        | }
    ✓ if (chaine == chaineInverse) {
        |     print("La chaîne est un palindrome.");
    ✓ } else {
        |     print("La chaîne n'est pas un palindrome.");
        | }
    }
}
```

Exercice 7 :

On commence par déclarer la liste donnée, on déclare une nouvelle liste « pairs », on utilise une boucle qui parcourt la liste donnée et on fait une

vérification pour savoir si le nombre est pair, si c'est le cas on rajoute le nombre à la liste pairs, sinon on ne fait rien. A la fin on affiche la liste.

```
void main() {  
    List<int> a = [1, 4, 9, 16, 25, 36, 49, 64, 81, 100];  
    List<int> pairs = [];  
  
    for (int nombre in a) {  
        if (nombre % 2 == 0) {  
            pairs.add(nombre);  
        }  
    }  
  
    print(pairs);  
}
```

Résultats :

```
[4, 16, 36, 64, 100]
```

Exercice 9 :

Cette fois-ci, nous allons ajouter une bibliothèque en plus : dart:math

Cette bibliothèque permettra d'avoir accès à la fonction random qui nous sera utile pour ce jeu de pierre papier ciseau.

On démarre avec une liste de choix possibles : pierre / papier / ciseau.

Ensuite on demande à l'utilisateur de choisir parmi les trois (je fais au plus simple donc je ne fais aucune vérification) mais je force quand même les lettres en minuscules pour pas qu'il y ait des problèmes de match. Ensuite on attribue le choix à la variable joueur.

Ensuite on déclare un objet random qui servira d'adversaire.

On indique que la variable ordi va récupérer la valeur du choix de random, random.nextInt va en fait faire choisir un nombre entre 0 et 2 car le 3 indique en fait une limite à ne pas égaler ni franchir. Donc le robot va prendre un indice aléatoire, et par conséquent avoir un choix aléatoire.

On affiche par la suite au joueur ce qu'à choisi l'ordi puis on fait les vérifications :

Si le joueur et l'ordi ont eu le même choix : on affiche égalité

Sinon si le joueur a fait pierre et l'ordi ciseau ou joueur a fait papier et l'ordi a fait pierre ou le joueur a fait ciseau et l'ordi papier (On utilise || comme OU) alors le joueur gagne et dans n'importe quel autre cas il perd.

```
import 'dart:io';
import 'dart:math';

Run | Debug | Profile
void main() {
  List<String> choixPossibles = ['pierre', 'papier', 'ciseaux'];

  print("Votre choix (pierre, papier ou ciseaux) :");
  String joueur = stdin.readLineSync()!.toLowerCase();

  Random random = Random();
  String ordi = choixPossibles[random.nextInt(3)];

  print("L'ordinateur a choisi : $ordi");

  if (joueur == ordi) {
    print("Égalité !");
  } else if (
    (joueur == 'pierre' && ordi == 'ciseaux') ||
    (joueur == 'papier' && ordi == 'pierre') ||
    (joueur == 'ciseaux' && ordi == 'papier')
  ) {
    print("Vous avez gagné !");
  } else {
    print("L'ordinateur a gagné !");
  }
}
```

Résultat :

```
Votre choix (pierre, papier ou ciseaux) :
papier
L'ordinateur a choisi : pierre
Vous avez gagné !
```

Exercice 9 :

